

Nimbus — Architecture in Diagrams

Version: 1.5 **Author:** Sujal Kumar Singh **Last updated:** 2026-08-12

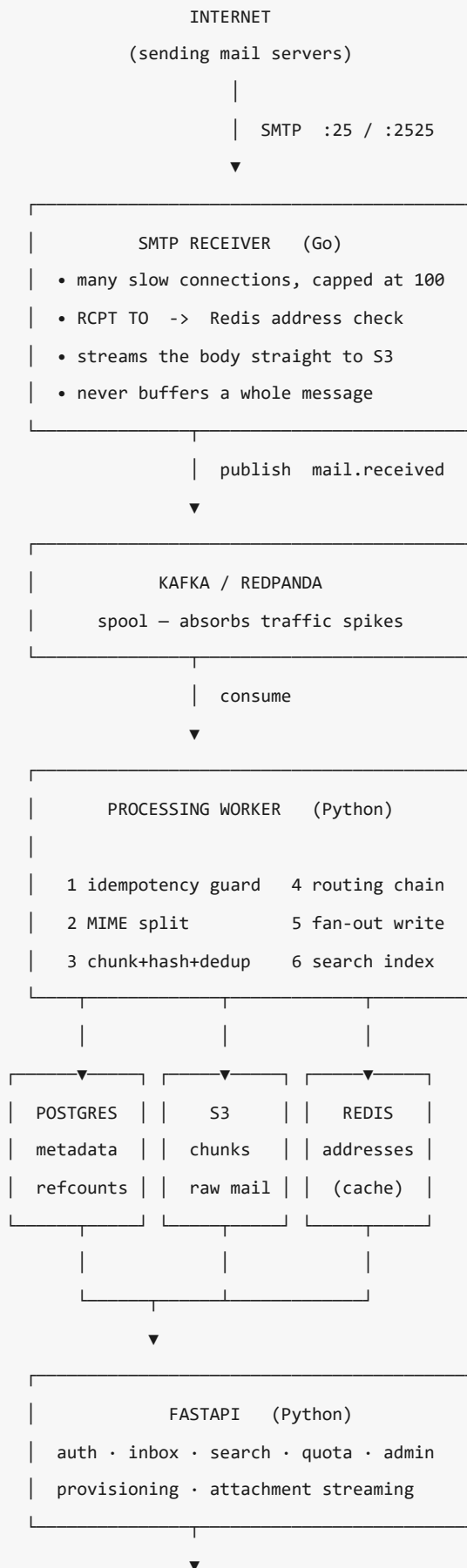
Every part of the system, drawn. Almost no prose — each diagram gets one line saying what it shows and a few points that matter.

- Plain-English explanation → `OVERVIEW.md`
- Exact columns, endpoints, steps → `HLD.md`

Contents

#	Diagram
1	The whole system
2	Deployment: local vs AWS
3	Data model — tenancy
4	Data model — storage and messages
5	How one attachment is physically stored
6	What dedup actually saves
7	Inbound mail, end to end
8	The accept / reject deadline
9	The dedup decision
10	Fan-out: one message, many inboxes
11	Surviving a repeated queue message
12	Routing: who really gets it
13	RefCount lifecycle
14	Garbage collection: how the bytes actually come back
15	Reading an attachment (range requests)
16	Search: text to SQL
17	Snooze: a predicate, not a timer
18	Auth: two doors
19	Provisioning without double-creating
20	Build order

1. The whole system



REACT UI (TypeScript)

BACKGROUND WORKERS (nothing waits on these)

GC SWEEP (Python)
refcount 0, garbage 24h
-> delete chunk from S3

There is no snooze worker. Snooze is a predicate – diagram 17.

Why the split: Go handles many slow connections cheaply. Python does the thinking, where nothing is latency-critical. Kafka sits between them so a spike never drops mail.

2. Deployment: local vs AWS

LOCAL (docker compose)

AWS (production)

postgres	container	→	RDS
redis	container	→	ElastiCache
minio	container	→	S3
redpanda	container	→	Redpanda in Docker on EC2
app	containers	→	EC2 t3.small
–			Route 53 MX record

port 2525 → port 25

▲
└─ AWS blocks this by default.
Unblock request takes DAYS.
File it before writing code.

The same boto3 client talks to MinIO and to S3 with no code change.

Why Redpanda on EC2 and not MSK: MSK's cheapest configuration is around \$100/month. Not justified at this scale.

3. Data model — tenancy

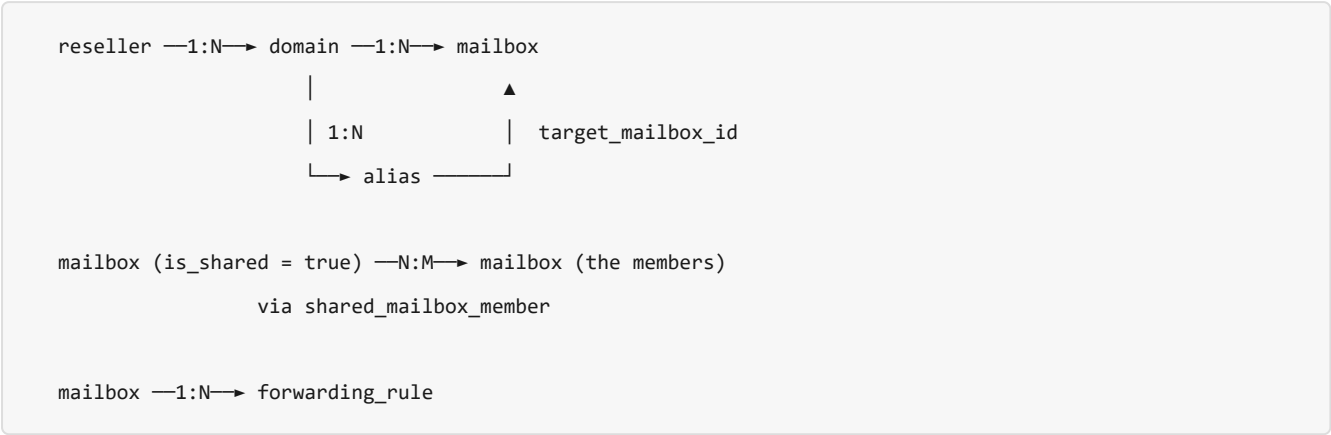


Table	Key columns
reseller	id, name, api_key_hash, webhook_url
domain	id, reseller_id, name, verified, catch_all_mailbox_id
mailbox	id, domain_id, local_part, password_hash, quota_bytes, is_shared, used_bytes
alias	id, domain_id, local_part, target_mailbox_id
forwarding_rule	id, mailbox_id, forward_to_addr, keep_copy
shared_mailbox_member	shared_mailbox_id, member_mailbox_id

4. Data model — storage and messages

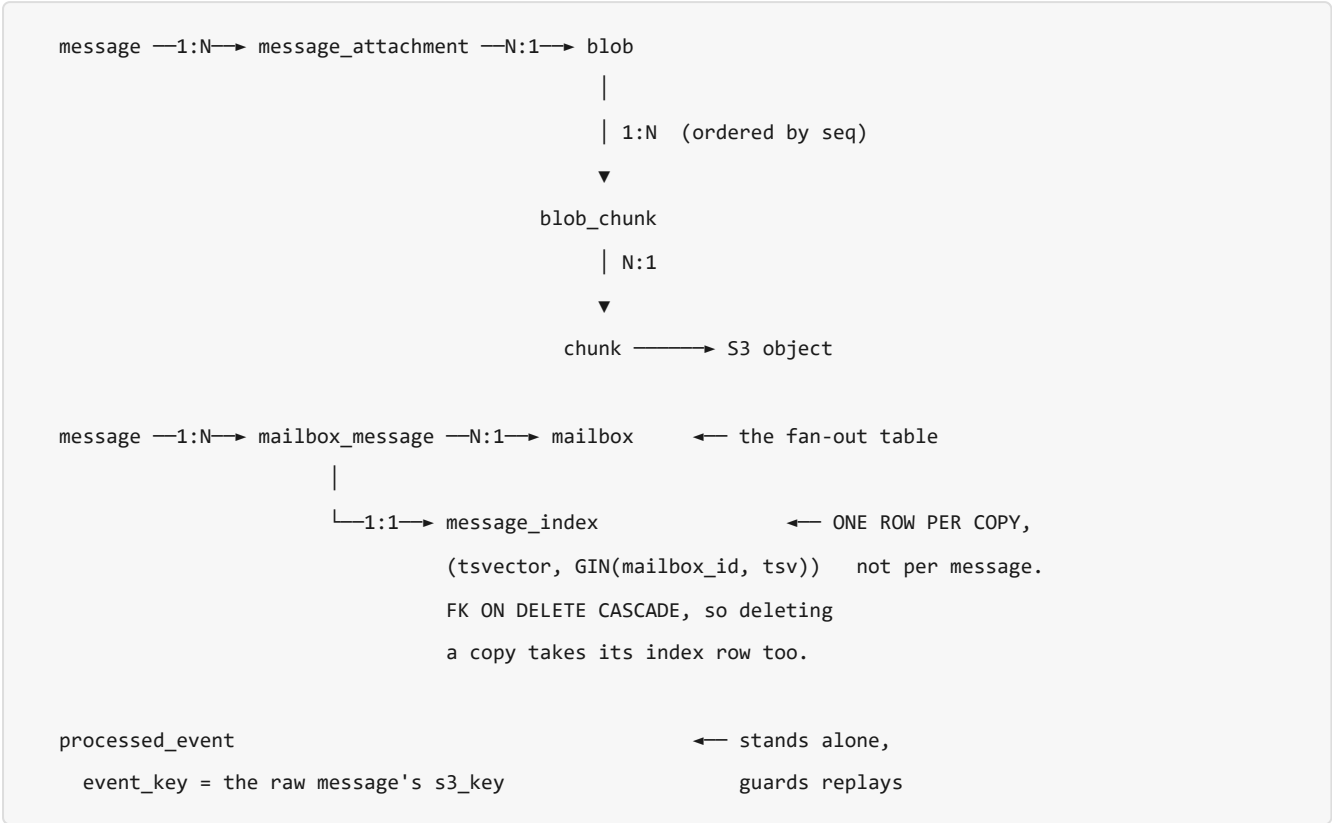


Table	Key columns	Note
blob	(reseller_id, hash), size_bytes, chunk_count, refcount	Key includes reseller_id — dedup is per customer , never global
chunk	(reseller_id, hash), size_bytes, s3_key, refcount	
blob_chunk	reseller_id, blob_hash, seq, chunk_hash	seq is the reassembly order
mailbox_message	(mailbox_id, message_id), folder, is_read, snooze_until	~40 bytes per row
processed_event	event_key PK, message_id, processed_at	See diagram 11

5. How one attachment is physically stored

ONE EMAIL WITH ONE 10 MB ATTACHMENT

```
message id = 501
|
└ message_attachment filename="Q3.pdf" content_type=application/pdf
    |
    ▼
    blob hash=3a7f9c.. size=10 MB chunk_count=3 refcount=40
        |
        | blob_chunk, in order
        |
        seq 0 → chunk 8e1d.. → s3://chunks/{reseller}/8e/8e1d... [ 4 MB ]
        seq 1 → chunk b704.. → s3://chunks/{reseller}/b7/b704... [ 4 MB ]
        seq 2 → chunk 2fa9.. → s3://chunks/{reseller}/2f/2fa9... [ 2 MB ]
```

The reseller id is IN the key, not decoration. Dedup is scoped per customer (HLD §12), so the same bytes at two resellers are two objects. A shared path would put back the cross-tenant leak the composite primary key removes.

A SECOND EMAIL ARRIVES WITH THE SAME ATTACHMENT

```
message id = 502
└ message_attachment → blob 3a7f9c.. refcount 40 → 80
    |
    └ NO new chunks written
        NO new S3 uploads
        0 extra bytes on disk
```

The fingerprint is the filename. Same bytes always land on the same row, so writing the same file twice is harmless by construction — no lock required for correctness.

6. What dedup actually saves

One 25 MB slide deck sent to 40 people

NORMAL MAIL SERVER

NIMBUS

[25 MB] [25 MB] [25 MB] [25 MB]

[25 MB] ← the bytes, once

[25 MB] [25 MB] [25 MB] [25 MB]

▲

[25 MB] [25 MB] [25 MB] [25 MB]

1

[25 MB] [25 MB] [25 MB] [25 MB]

| 40 rows x ~40 bytes

[25 MB] [25 MB] [25 MB] [25 MB]

```
| = 1.6 KB total
```

[25 MB] [25 MB] [25 MB] [25 MB]

1

[25 MB] [25 MB] [25 MB] [25 MB]

1

[25 MB] [25 MB] [25 MB] [25 MB]

1

[25 MB] [25 MB] [25 MB] [25 MB]

1

[25 MB] [25 MB] [25 MB] [25 MB]

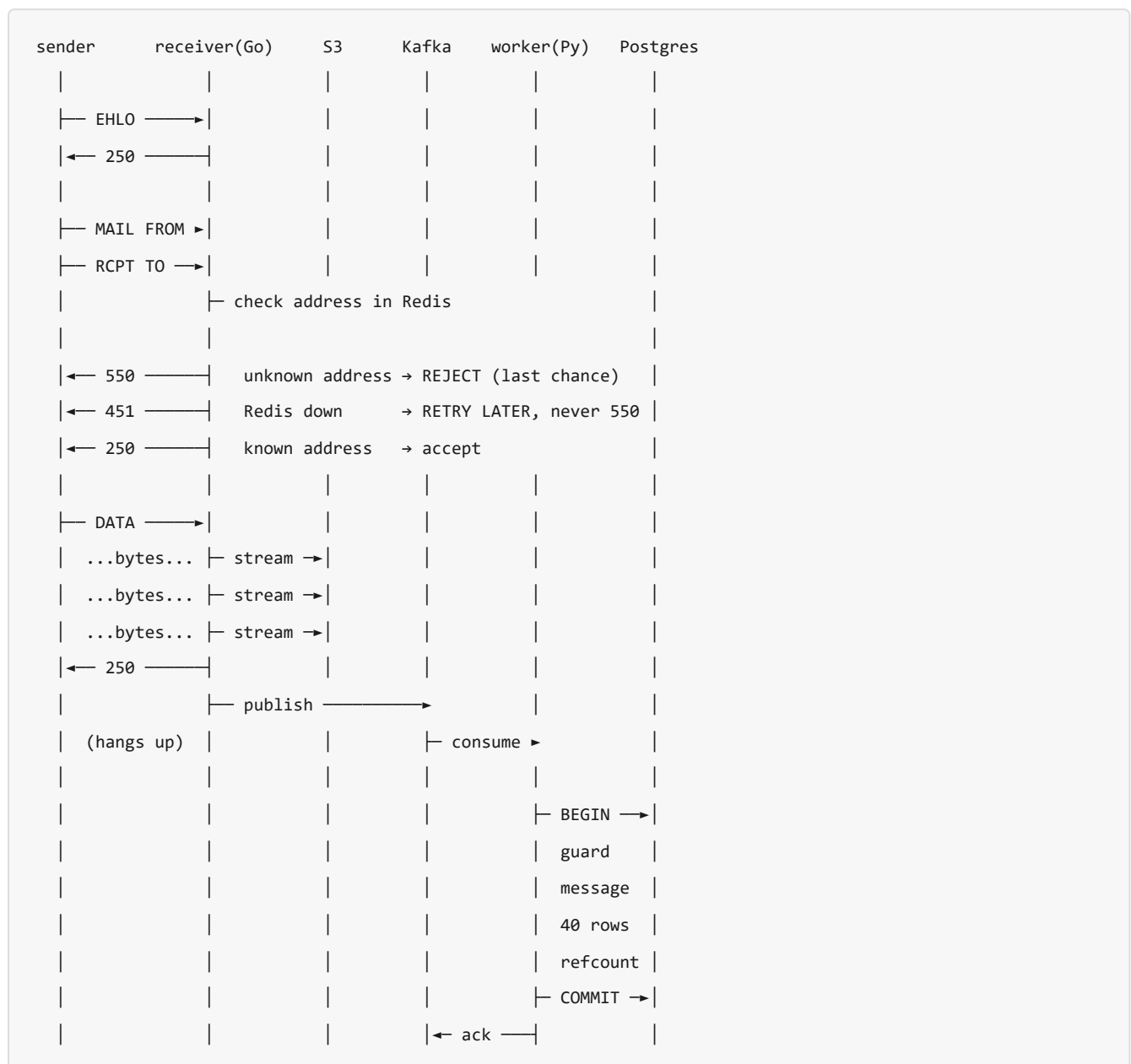
ON DISK: 1000 MB

ON DISK: 25 MB + 1.6 KB

40x less

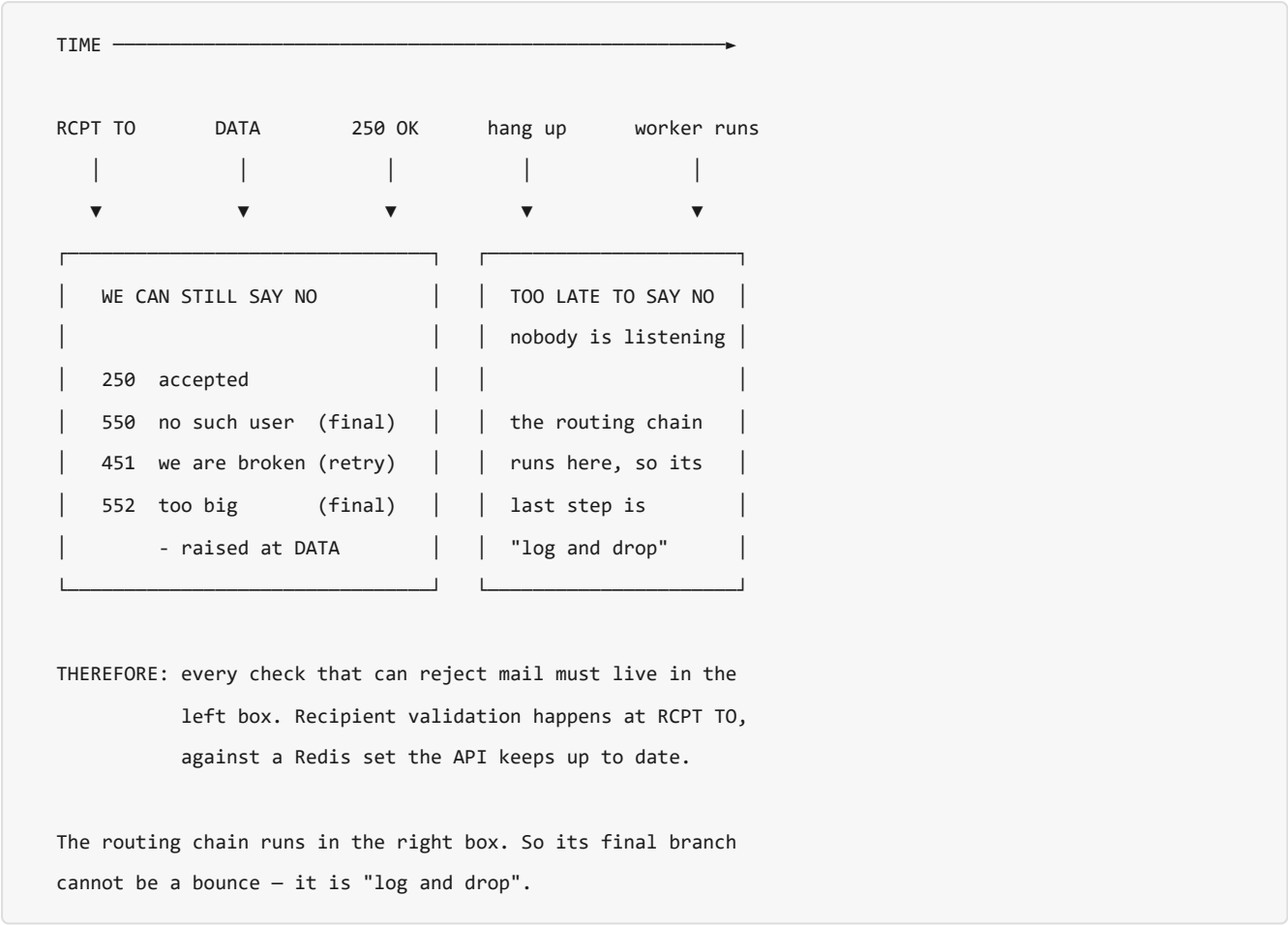
Where it does not help: sending. SMTP transmits every byte to every recipient no matter what. Dedup is a *storage* win only. That is why Nimbus is receive-side.

7. Inbound mail, end to end



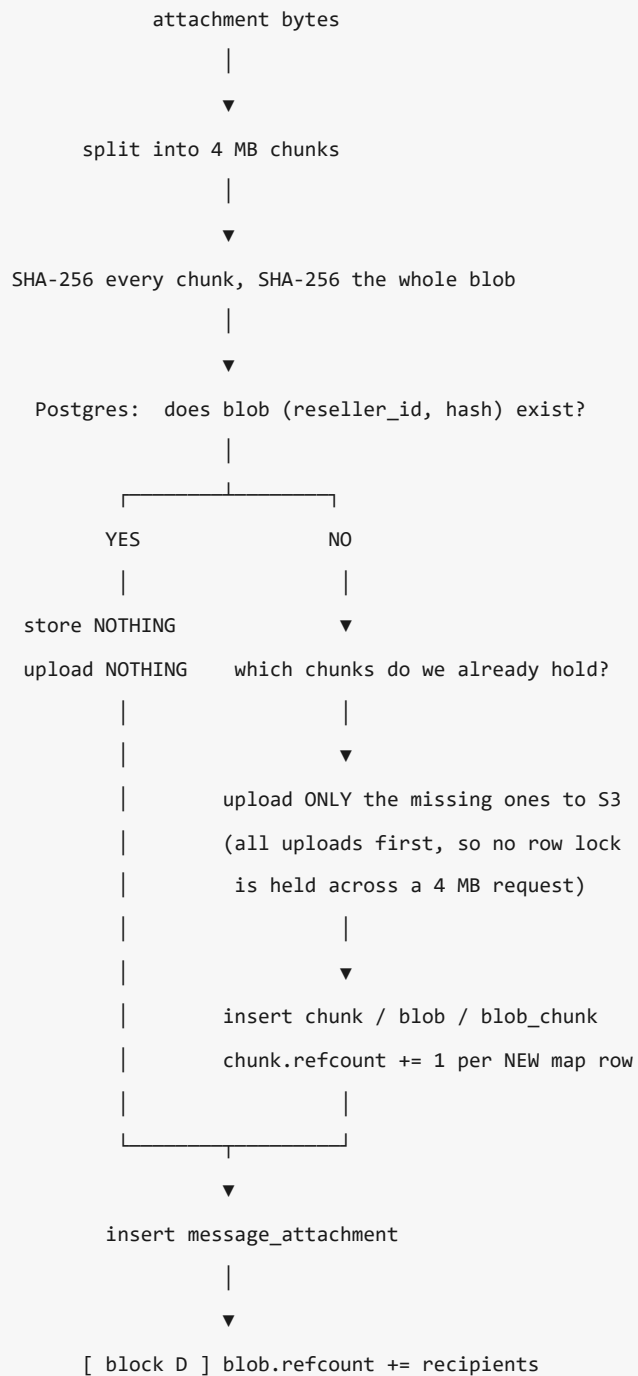
- Memory in the receiver stays flat. The message is never held whole.
- Once the event is on Kafka the mail is safe. A worker crash just replays it.
- Everything the worker writes lands in **one transaction**. All of it, or none.

8. The accept / reject deadline



This is the correction that shaped the design. Sending a real bounce email would mean outbound SMTP, which is an explicit non-goal.

9. The dedup decision

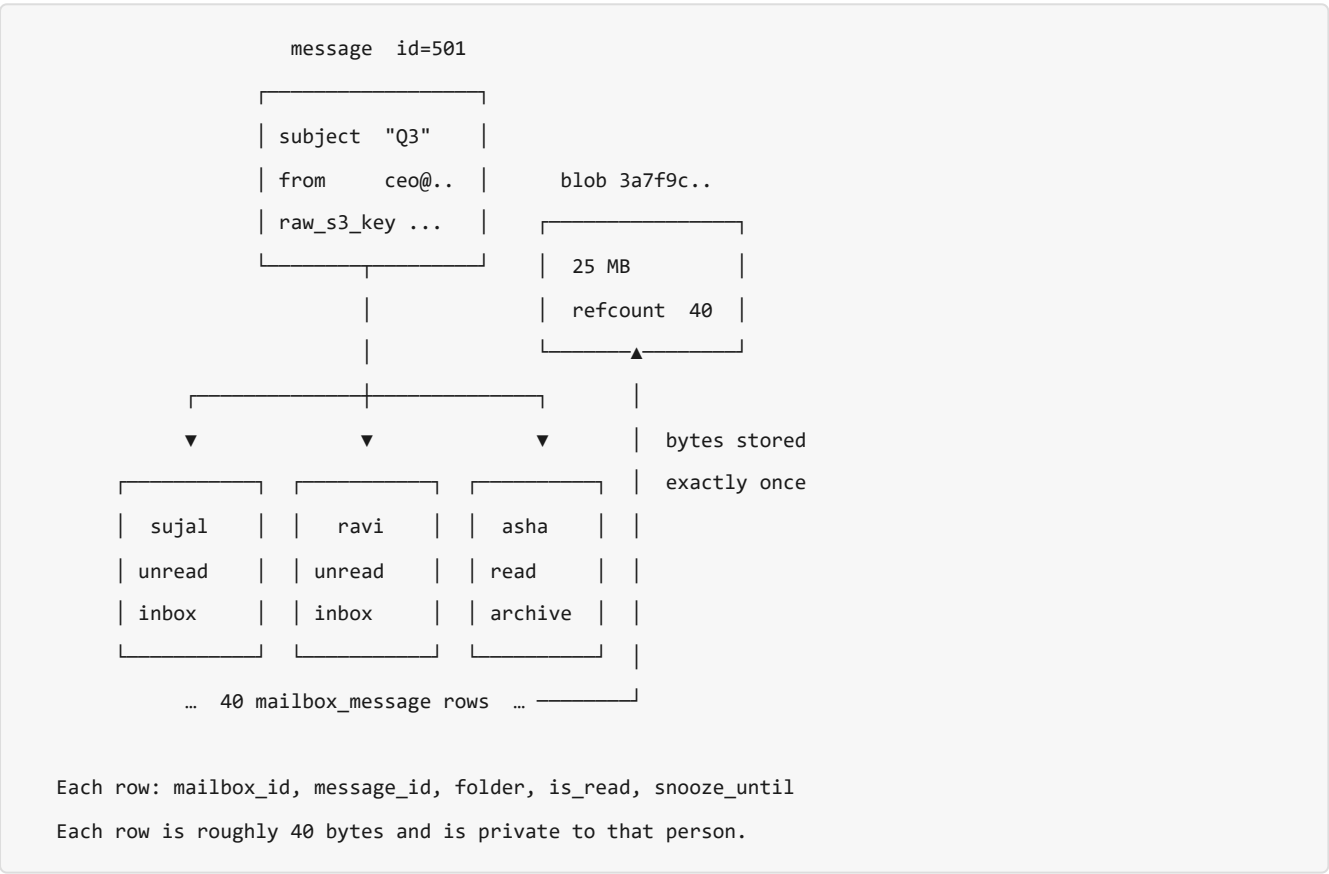


There is no Redis cache and no Redis lock here, and both were planned. Dropped deliberately during block C:

Planned	Why it is gone
Redis "does this blob exist?" cache	Nothing invalidates it when GC deletes a blob, so a stale hit claims bytes exist after they are gone. It replaces one primary-key lookup inside a transaction we are already in — about 0.3 ms — with a consistency hazard.
Redis lock on the hash	<code>ON CONFLICT DO NOTHING</code> already makes the losing writer a no-op. Verified with two interleaved transactions: 1 blob, 2 chunk-map rows, refcount 1 each — no double count, no lost count. The lock only ever saved a duplicate upload, and content addressing makes that harmless.

Postgres is the sole source of truth for what is stored.

10. Fan-out: one message, many inboxes



Per-person state (read, folder, snoozed) lives on the row. Shared content lives once.

11. Surviving a repeated queue message

Kafka delivers **at least once**. The same event will arrive twice eventually.

WITHOUT A GUARD

event arrives
insert 40 rows
refcount 0 → 40
crash before ack
|
▼
event is redelivered
insert 40 rows AGAIN
refcount 40 → 80
|
▼
inbox shows it twice
refcount permanently wrong
GC can never free the blob

WITH THE GUARD

CASE 1 crash BEFORE commit

transaction rolls back
nothing was written
event replays and does
the work cleanly
✓ exactly one copy

CASE 2 commit ok, crash BEFORE ack

replay inserts processed_event
with ON CONFLICT DO NOTHING
→ RETURNING gives back nothing
process_event() returns False
event is acked, nothing changes
✓ exactly one copy

⚠ **ON CONFLICT DO NOTHING , not a caught primary-key collision** — this diagram used to say the replay collides and the transaction aborts. That is the design we rejected, and `pipeline.py` says why in its own comment: in Postgres **any** error aborts the whole transaction, and every statement after it still has to run inside that transaction. So a collision here would poison the work we are about to do rather than skip it neatly. The guard has to be a no-op, not an exception. Anyone building block G from the old version of this diagram would have reintroduced the abort.

THE TRANSACTION

BEGIN

```
INSERT processed_event(event_key = s3_key)  ← first line, always
INSERT message
INSERT mailbox_message × N                  ← N = rows ACTUALLY created
UPDATE blob.refcount += N
UPDATE mailbox.used_bytes
SAVEPOINT —————
  INSERT message_index × N                  ← block F. NOT all-or-nothing
RELEASE / ROLLBACK TO SAVEPOINT —————    with the rest – see below
COMMIT
```

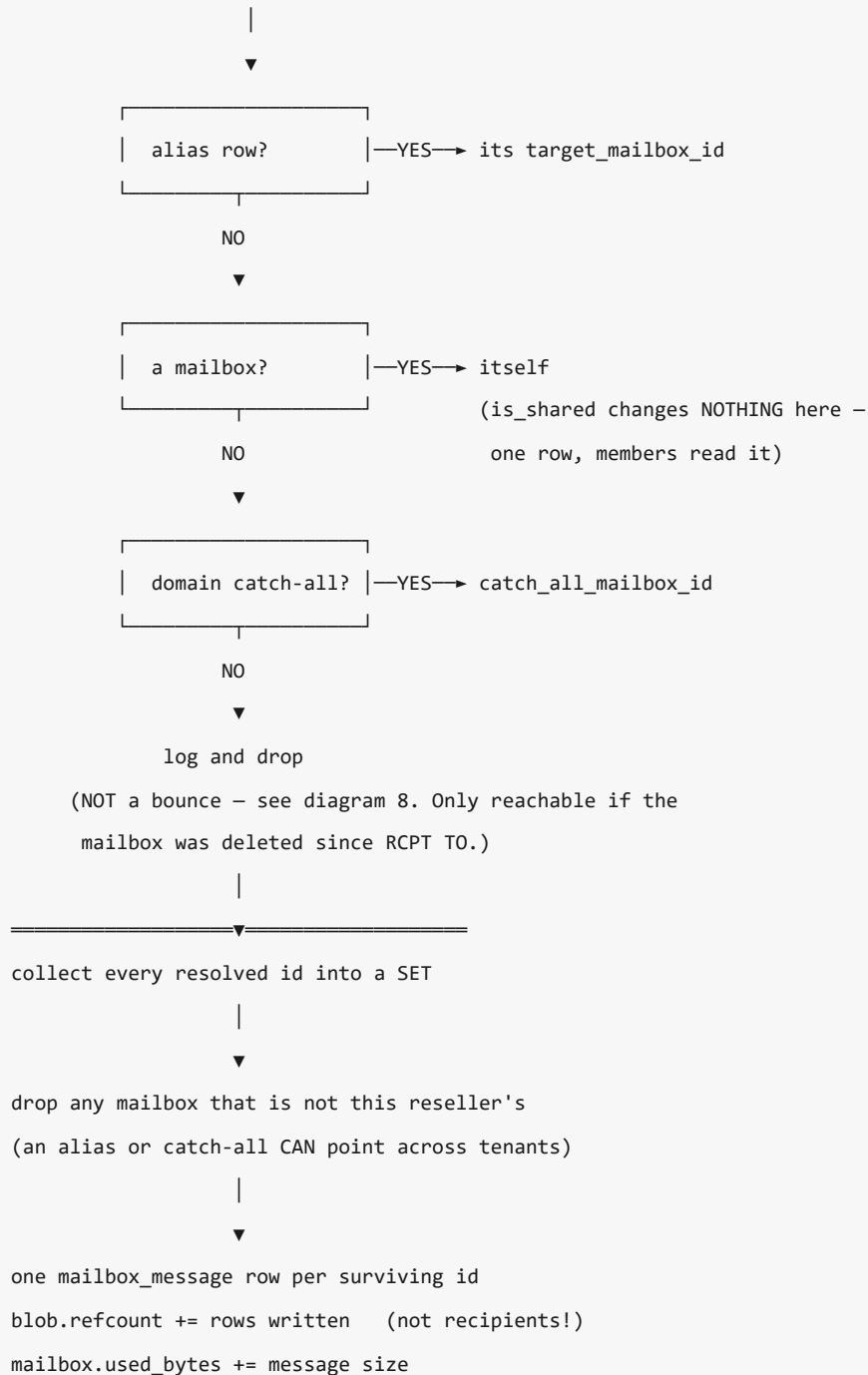
Put the guard first and the whole thing becomes idempotent for free.

Everything above the savepoint is all-or-nothing. The index write is not, on purpose. By the time it runs, the bytes are in S3 and the mail is delivered. Letting a search-index problem roll all of that back would leave the Kafka offset uncommitted and the message redelivered forever — one bad email would stop that partition. Unsearchable-but-delivered is recoverable; undelivered is not. Proven in practice: the first live run of block F failed on a Postgres type mismatch and the log read `delivered ... to 2 mailbox(es), indexed 0` — the mail arrived anyway.

The exception is a **transient** failure — a deadlock, a serialization failure, a dropped connection (SQLSTATE class `40`, `08`, `57`). Those say nothing about this message, so they are re-raised and the whole event is retried rather than committed unindexed.

12. Routing: who really gets it

every recipient address, already accepted at RCPT TO



A SET, not a list. `sales@` aliased to Alice plus `alice@` directly is two recipients and one delivery. Two rows would double `blob.refcount`, and the extra never comes back down, so GC could never free that blob.

Two branches this diagram used to have, and why they are gone:

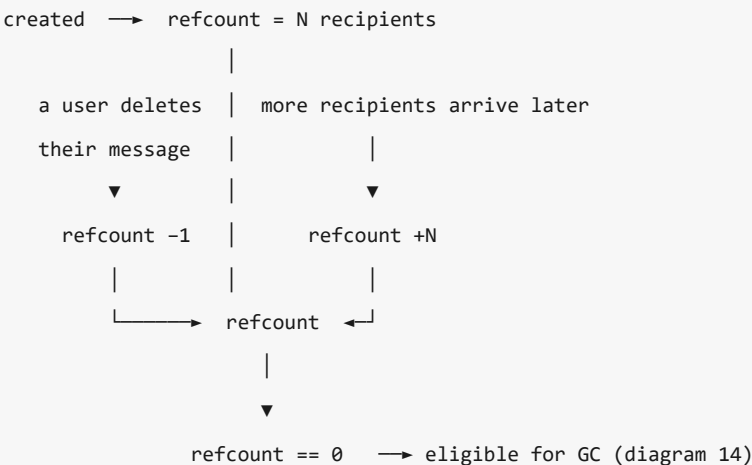
Removed	Why
<code>is_shared</code> → expand to every member	A shared mailbox gets ONE copy; <code>shared_mailbox_member</code> grants READ access. K rows, K refcounts and K quota charges for one email, plus a private read state, is the opposite of what a <code>support@</code> inbox needs.
<code>forwarding_rule</code> → send onward	Needs an outbound SMTP client, an explicit non-goal (HLD §4). Honouring <code>keep_copy = false</code> without one would delete the only copy and send nothing.

13. Refcount lifecycle

TWO COUNTERS, TWO MEANINGS

`blob.refcount` = how many `mailbox_message` rows reach this blob
`chunk.refcount` = how many `blob_chunk` rows point at this chunk

BLOB



CHUNK

`chunk 8e1d..` is used by blob A and blob B `refcount = 2`

`blob A deleted` → `refcount = 1` → KEEP, B still needs it
`blob B deleted` → `refcount = 0` → DELETE from S3

The trap: counting messages instead of `mailbox_message` rows under-counts, and frees a blob that two people are still reading.

14. Garbage collection: how the bytes actually come back

THE SWEEP – THREE PHASES, IN THIS ORDER

1. SELECT ... FOR UPDATE SKIP LOCKED, then a SEPARATE DELETE of message rows that have NO mailbox_message rows left

| (cascades message_attachment AND message_index)
| TWO statements – see the warning below

▼

2. find blobs where refcount = 0

AND refcount_zeroed_at older than 24 hours

|

▼

chunk.refcount -= count(*) of that blob's blob_chunk rows

| count(*), NOT 1 – one blob can list the
| same chunk twice (8 MB of zeros)

▼

delete blob (blob_chunk cascades with it)

|

▼

3. DELETE FROM chunk WHERE refcount = 0 RETURNING s3_key

|

▼

delete those S3 objects → COMMIT

keys come from RETURNING, never from the candidate list,
and the object delete is INSIDE the transaction

WHY PHASE 1 EXISTS (found while verifying block D, proven live)

Skip it and phase 2 raises:

ForeignKeyViolationError: update or delete on table "blob"
violates constraint "message_attachment_reseller_id_blob_hash_fkey"

message_attachment references blob with NO ACTION, and nothing else ever deletes a message row. So refcount reaches 0 and the blob becomes PERMANENTLY uncollectable – the dedup engine would never reclaim a byte.

The FK is deliberately NOT ON DELETE CASCADE: that would let a blob delete strip a live message's attachment list. Failing loudly is what proves the ordering is right.

WHY PHASE 2 MUST PRECEDE PHASE 3 (a second reason, easy to miss)

pipeline._store_attachment short-circuits on the BLOB row:

```
known = SELECT hash FROM blob WHERE ...  
if known is None: _store_new_blob(...) ← never runs
```

A surviving blob whose chunks were already collected NEVER reaches the chunk-level check, never uploads, never notices. It just serves a corrupt attachment. Silently.

WHY PHASE 1 NEEDS TWO STATEMENTS

A single DELETE ... WHERE NOT EXISTS (SELECT 1 FROM mailbox_message) is UNSAFE. Blocked on the FK lock by a concurrent delivery, it deletes the parent ANYWAY when it unblocks: EvalPlanQual re-checks conditions against the ROW, and the row was only key-share locked, never updated – so the subquery keeps its stale snapshot.

The cascade then removes the copy that just committed. Mail gone, and the trigger drops blob.refcount and used_bytes with it.

```
SELECT ... FOR UPDATE SKIP LOCKED ← take the lock  
DELETE ... AND NOT EXISTS (...) ← fresh snapshot
```

The lock is the stronger half, and saying only "fresh snapshot" undersells it. An FK insert into mailbox_message takes FOR KEY SHARE on the parent message row, which CONFLICTS with FOR UPDATE (verified with two live sessions: the insert blocks). So while the lock is held, no child row can appear at all.

One window survives that, and it is what the repeated NOT EXISTS is for: an insert that COMMITTED after the SELECT's snapshot but before the scan reached the row. Snapshot says orphan, lock is free, row is returned – and the DELETE refuses it.

That cannot spin. The next SELECT's snapshot is strictly newer than this DELETE's, so a refused row is still non-orphan and is not selected again. Phase 2 needs an explicit `break` only because its failure path rolls a batch back UNCHANGED.

--dry-run DOES THE DELETES, THEN ROLLS THE WHOLE SWEEP BACK

It cannot work per-phase, and the first version proved it:

```
phase 1  select orphans → ROLLBACK → report N
phase 2  needs message_attachment GONE to see any blob
        ...but phase 1 was just undone           → reports 0
phase 3  needs phase 2's chunks zeroed           → reports 0
```

Two of three numbers were ALWAYS zero, on the one command an operator runs to decide whether GC is worth scheduling – in a system where nothing schedules GC automatically.

BEGIN → phase 1 → phase 2 → phase 3 (skip S3) → ROLLBACK

Cost: locks held for the whole sweep, not one batch. Acceptable for a command typed deliberately, and exactly why the REAL sweep commits per batch instead.

A FAILED S3 DELETE MUST COMMIT ANYWAY

Partial failure = some keys gone, some not.

```
ROLLBACK  every chunk row survives at refcount 0, but the keys
          that DID delete are gone. Next delivery of those bytes
          finds the row, skips the upload, refcount → 1 on a
          chunk pointing at nothing.           ← CORRUPTION
COMMIT    rows gone, failed keys orphaned.    ← COSTS MONEY
```

Commit, then exit non-zero. Cron must not report success while objects are left unreferenced and only a log line names them.

WHAT ACTUALLY MAKES THE RACE SAFE – THE FOREIGN KEY, NOT THE WAIT

t = 0 mailbox A deletes its copy blob refcount 1 → 0
t = 0 mailbox B is MID-DELIVERY of the very same file
t = 1s B commits blob refcount 0 → 1

Whichever gets there first, the FK serialises them:

CASE 1 B is uncommitted when GC tries to delete

GC: DELETE FROM blob WHERE ... AND refcount = 0
 blocks on B's key-share lock (message_attachment → blob)
B: commits, refcount is now 1
GC: unblocks, re-checks refcount = 0, matches NOTHING ✓ SAFE

CASE 2 GC is uncommitted when B tries to insert

B: INSERT message_attachment blocks on GC's row lock
GC: commits, blob is gone
B: ForeignKeyViolationError – LOUD ✓ SAFE
 transaction aborts, Kafka redelivers, retry re-uploads

This works at ANY grace period, including zero.

⚠ **This panel used to argue the opposite, and its own example disproved it.** It said the 24-hour wait was what made the race safe — but B is uploading a file *we already hold*, so the blob is old, and the age check the old design used (`created_at`) passed immediately. The worked example was the exact case the mechanism failed on.

So what is the grace period for? An operator recovery window. It is 24 hours between a refcount reaching zero and the bytes becoming unrecoverable, in which a human can notice that it reached zero when it should not have. That is worth having — it is just not what the previous version claimed, and saying so is what stops the next reader deleting it as redundant.

And it now measures the right clock. `refcount_zeroed_at` , the moment the blob became garbage — not `created_at` , which is when the bytes were written and never moves. Under the old column a blob stored a year ago whose last reference vanished a second ago was collectable on the very next sweep, while a blob written an hour ago was held for 23 more hours for nothing. Migration `9c3e5a1d7b42` .

The trade: one day of wasted disk on genuinely dead blobs, in exchange for a window to catch a GC bug before it is permanent. Cheap price.

15. Reading an attachment (range requests)

```
GET /v1/messages/501/attachments/3a7f9c..
```

```
Range: bytes=4194304-8388607
```

|

▼

```
look up blob_chunk ORDER BY seq
```

seq 0	chunk 8e1d..	bytes	0 - 4 194 303	
seq 1	chunk b704..	bytes	4 194 304 - 8 388 607	← only this one
seq 2	chunk 2fa9..	bytes	8 388 608 - 10 485 759	

|

▼

```
fetch ONLY the needed slice of b704.. from S3 → stream to client
```

```
206 Partial Content
```

```
Content-Range: bytes 4194304-8388607/10485760
```

The slice is pushed down to S3 as its own Range header, so serving 1 KB moves 1 KB – not the whole 4 MB chunk it happens to live in.

```
ETag: "3a7f9c.."      Cache-Control: immutable
```

The bytes at a content hash can never change, so a re-open is a 304.

A FULL DOWNLOAD, NO RANGE HEADER

```
read chunk 0 → send → forget
```

```
read chunk 1 → send → forget
```

```
read chunk 2 → send → forget
```

MEMORY USED: one chunk at a time. Never the whole file.

A dropped connection can resume from the right chunk.

16. Search: text to SQL

```
"from:boss has:attachment before:2026-01-01 invoice"

|
▼ lift out the operators, leave the rest
[from:boss] [has:attachment] [before:2026-01-01] + "invoice"

|
▼ one flat set of filters – NOT a tree
Query(from_contains="boss", has_attachment=True,
      before=2026-01-01, free_text="invoice")

|
▼ compile to SQL
SELECT m.id, m.subject, m.from_addr, mm.received_at, mm.folder, ...
FROM message_index mi
JOIN mailbox_message mm ON mm.mailbox_id = mi.mailbox_id ← BOTH columns
                      AND mm.message_id = mi.message_id
JOIN message m ON m.id = mi.message_id
WHERE mi.mailbox_id = ANY($1) ← the READABLE SET, see below
      AND mi.tsv @@ websearch_to_tsquery('english', 'invoice')
      AND m.from_addr ILIKE '%boss%'
      AND EXISTS (SELECT 1 FROM message_attachment
                  WHERE message_id = m.id)
      AND mm.received_at < '2026-01-01'
ORDER BY mm.received_at DESC, mm.message_id DESC, mm.mailbox_id DESC
LIMIT $2

|
▼

Bitmap Index Scan on message_index_tsv_mailbox_idx
Index Cond: (mailbox_id = ANY(...) AND tsv @@ ...) ← BOTH, in one scan
```

- The parser is a pure function: string in, **filters** out — not an AST. Four operators that all combine with `AND` do not need a tree. Testable with no database at all.
- **GIN (mailbox_id, tsv), one table, not partitioned.** `mailbox_id` is inside the index so the tenant filter is applied during the scan, not rechecked after the join. Measured: 5.3 ms vs 16.5 ms for a `GIN (tsv)` index that read another tenant's 50,000 rows and threw them away. HLD §9.4 has the numbers.
- The mailbox ids come from `visibility.readable_mailboxes()` — **never** from the query string, and never from the JWT alone. See the warning below for why "the JWT's mailbox id" is not the same thing and would be a bug.

⚠ Three things this diagram used to say, each of which was a bug.

1. `JOIN message_index mi ON mi.message_id = m.id` — joining on `message_id` alone. `message_index` has one row per *delivered copy*, so for a reader who can see two copies of one message (their own, plus a shared mailbox's) that join is a cartesian product and every such result appears twice. Both key

columns, always.

2. `WHERE mm.mailbox_id = ANY($1)` — filtering the wrong table. Same rows, but the condition can no longer be pushed into the GIN index, which is the entire point of putting `mailbox_id` in it.
3. `m.sent_at < ...` — filtering on the sender's own `Date:` header. It is attacker-supplied and `NULL` whenever it is missing or malformed, so every message with a broken date header would silently vanish from every date search. `received_at` is when we actually got it.

Also `plainto_tsquery` → `websearch_to_tsquery`: the former silently discards quoted phrases and negation, so `"exact phrase"` would match the same rows as `exact phrase`.

⚠️ = `$1` **would be wrong, and this diagram used to say it.** A single mailbox id is not what a reader can see. Block D delivers ONE copy to a shared mailbox and grants its members read access (§9.2), so search must filter on `nimbus.api.visibility.readable_mailboxes()` — your own mailbox plus every shared one you belong to — exactly like `GET /messages` does.

Filtering on the raw token subject would make a shared mailbox's mail **permanently unsearchable**: invisible to every member, and invisible to the mailbox itself, because shared mailboxes have `password_hash = NULL` and nobody can log in as one. It would be listable but not findable — two read surfaces disagreeing, silently.

17. Snooze: a predicate, not a timer

user snoozes a message until 09:00

|

▼

Postgres `mailbox_message.snooze_until = 09:00Z` ← one column, one write

|

|

... nothing runs. nothing ticks. ...

|

▼

every read of the inbox asks:

`WHERE snooze_until IS NULL OR snooze_until <= now()`

▲

evaluated by Postgres
└
at statement time

08:59:59 predicate false → hidden

09:00:00 predicate true → in the inbox

Nobody moved it. The clock did.

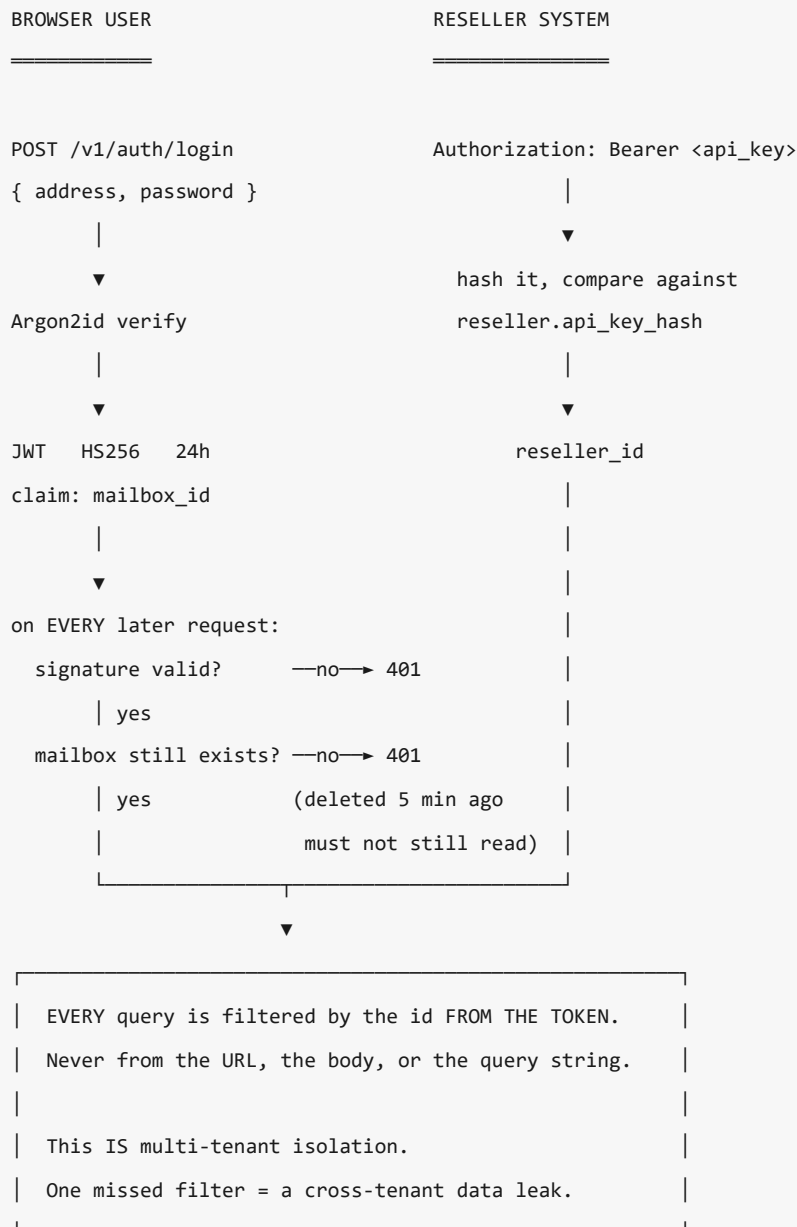
⚠️ **This diagram used to draw a Redis sorted set, a Go worker ticking every second, a ZREM race and an is_snoozed flag. All of it is gone**, and none of it was deleted for being slow — it was deleted for solving a problem snooze does not have.

Was drawn	Why it is gone
ZADD snooze_queue	snooze_until already held the time
A worker ticking every second	Nothing to do. Accuracy is now exact, not "within 1s"
ZREM ← only one node wins it	No node fires anything, so nothing needs winning
is_snoozed = true	Two columns for one state can disagree. Dropped
"bump the unread count"	No notification system exists, and §4 forbids outbound mail

Why Redis was the wrong store specifically. Redis in this system is a *cache* — `addresses.py` rebuilds it from Postgres at every startup, which is what makes it safe to lose. A snooze queue is the only record that a timer exists, so losing the volume hides mail for ever with no error and nothing to rebuild from. It also leaks: deleting a snoozed message removes the row and orphans the sorted-set entry, and nothing cleans that up.

Measured: on a 100k-message mailbox with 10% snoozed, the stored flag and the computed predicate produce identical query plans and identical buffer counts — 0.110 ms vs 0.103 ms. The flag bought nothing and cost a subsystem.

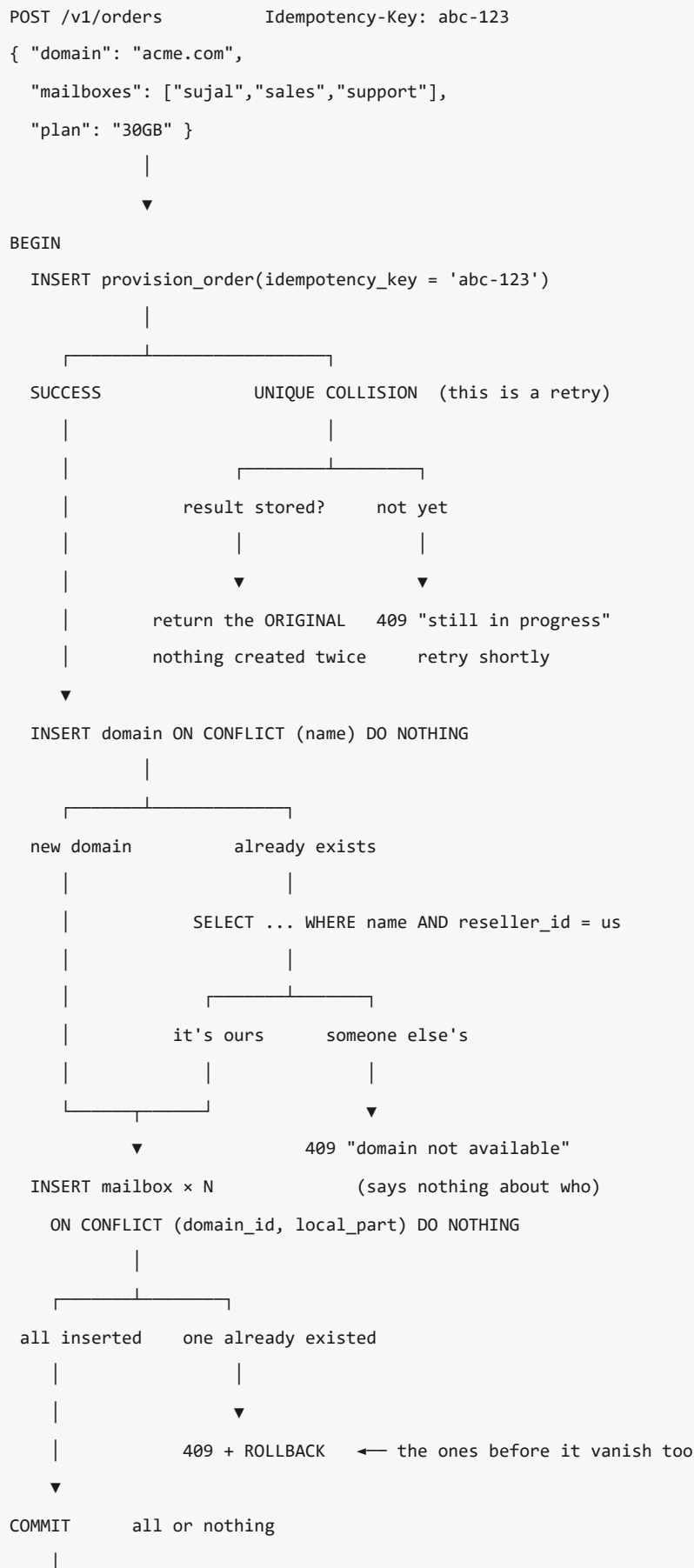
18. Auth: two doors



No refresh token in v1 — the token lasts 24 hours, then log in again.

Why the second check. A JWT is valid until it expires, whatever happened since. Without that lookup, deleting a mailbox leaves its holder reading mail for up to 24 more hours. It costs one primary-key read; the price is that the token is no longer verifiable offline. HLD §10.1 states that trade-off.

19. Provisioning without double-creating



▼

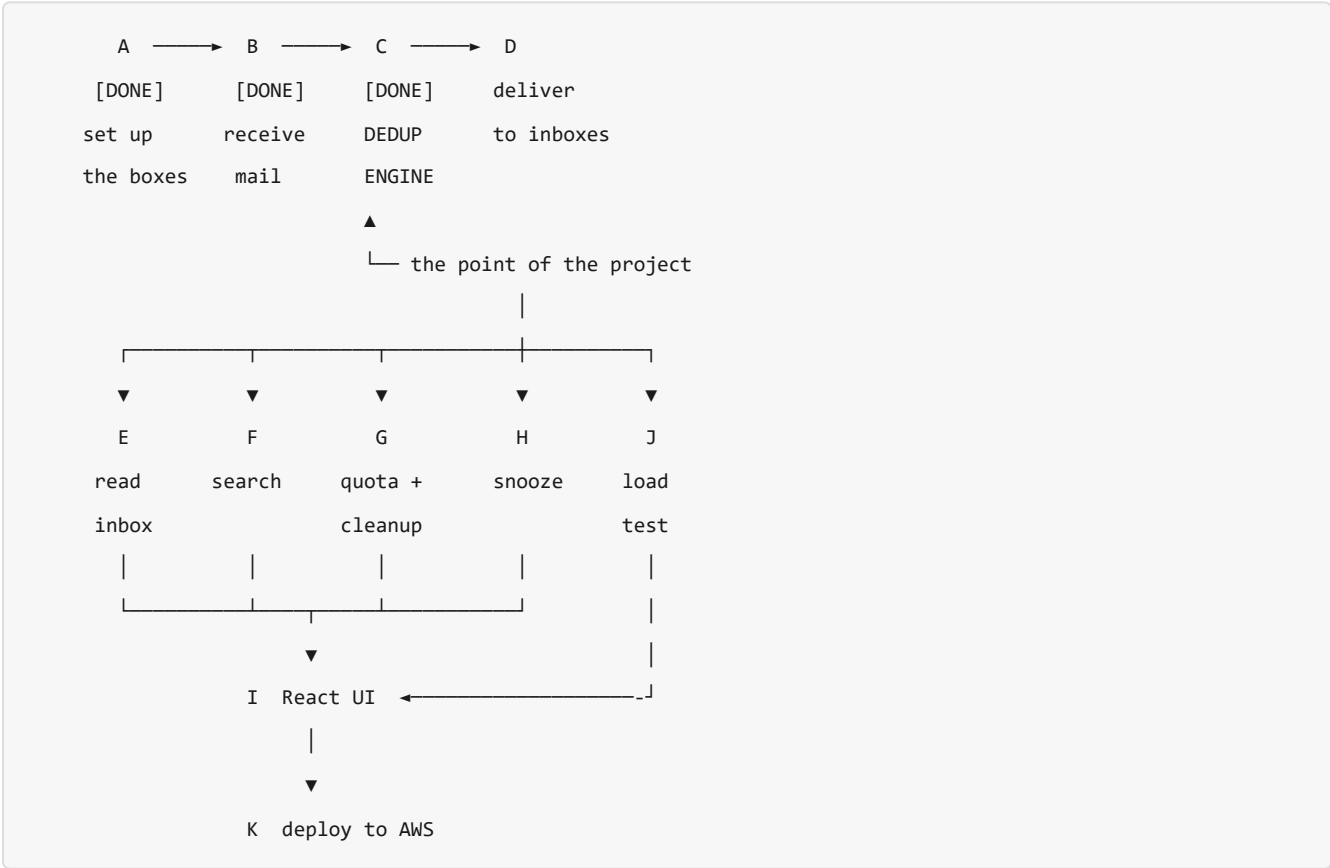
11

▼

▼

Why the domain branch: a reseller ordering more mailboxes on a domain they already own is normal. Only another tenant's domain is an error — and the error says nothing about who owns it, or an API key becomes a tool for probing other tenants.

20. Build order



Block	Needs	Risk	Note
A	—	low	Everything waits on this
B	A	medium	
C	A	high	The dedup engine
D	C	high	Routing + fan-out
E	D	medium	
F	D	medium	Parser can be written any time, no DB needed
G	D	high	GC is the subtle one
H	D	medium	
I	E F G H	medium	Shell can be built early against mock JSON
J	D	medium	Runs ~3.5 hours. Start early, build I while it runs.
K	J	low	Blocked on the AWS port 25 unblock

Critical path: A → B → C → D. After D, five things run independently.

Where to go next

You want	Read
Why any of this exists, in plain English	<code>OVERVIEW.md</code>
Exact columns, endpoints, steps	<code>HLD.md</code>
How we work on this project	<code>../CLAUDE.md</code>